

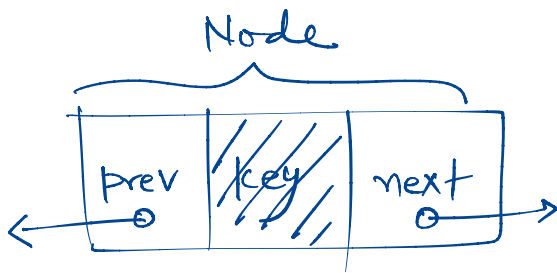
Linked Lists

- A data structure where objects are arranged in a linear order

↳ a pointer in each object determines the order

- Two types of Linked Lists (LL)
 - Singly Linked List
 - Doubly Linked List

- A doubly Linked List node has attributes : key : data



a node { next : Pointer to next obj.
Prev : Pointer to Previous obj

- For an element x :

$x.next$ points to the successor of x
 $x.prev$ " " " Predecessor of x

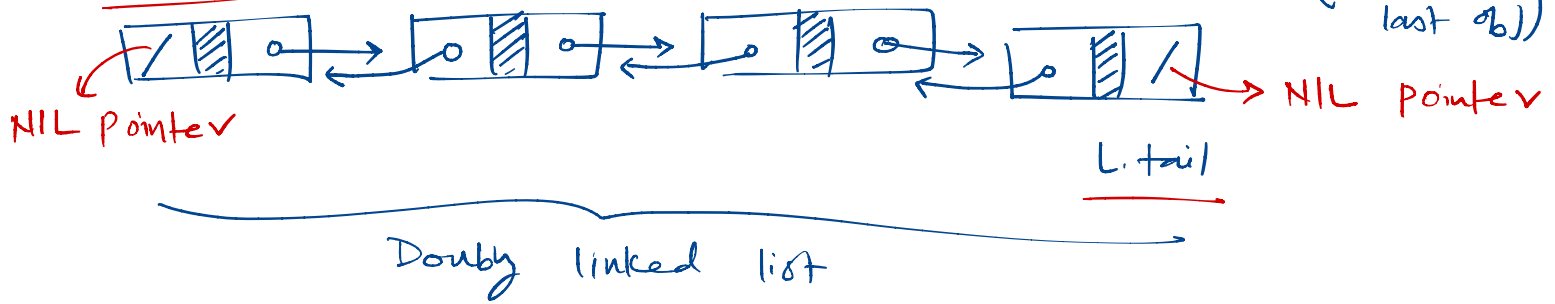
if $x.prev == NIL \Rightarrow$ No predecessor
 $\Rightarrow x$ is the first element of LL

↳ Head of the list

if $x.next == NIL \Rightarrow$ No successor
 $\Rightarrow x$ is the last element of LL

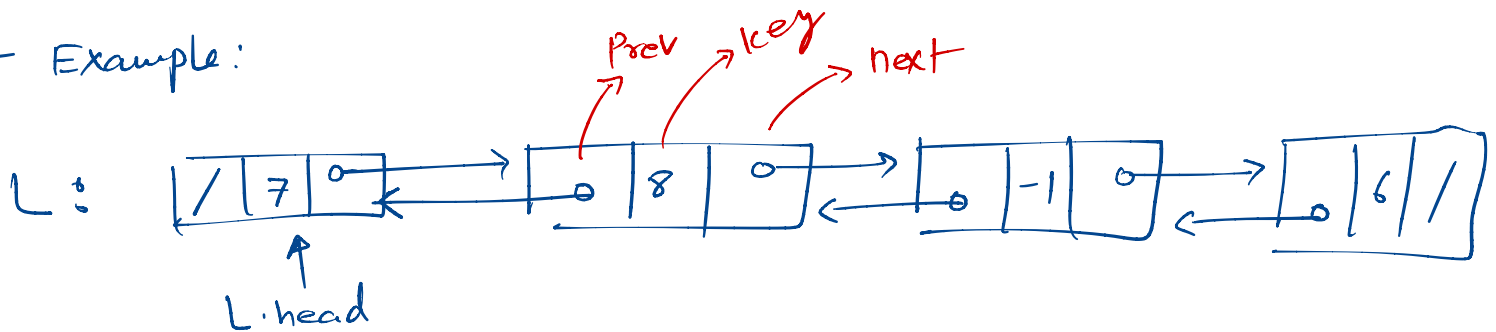
↳ Tail of the list

— Attributes of a doubly linked list : L.head (Points to first obj)
L.tail ("last" obj)



— A linked list may be : — singly or doubly
 — sorted or unsorted
 — circular or not

— Example:



— Searching a linked list

→ find the first element with a given key
 → Return NIL if not found

List-Search (L, k)

$x = L.head$

while $x \neq NIL$ and $x.key \neq k$

$x = x.next$

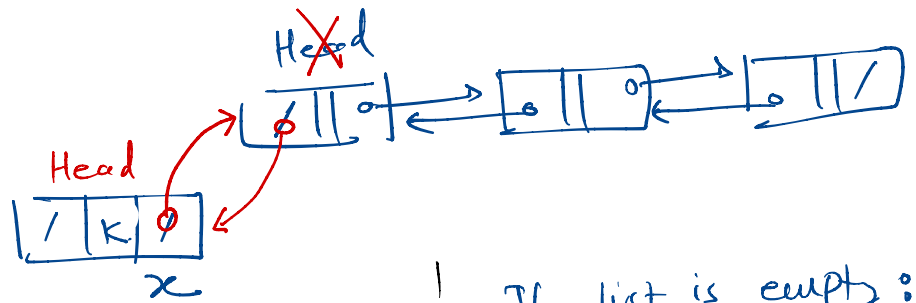
return x

→ $O(n)$

n : # of elements
in the list

— Inserting an element/node in a given linked list

a) Prepending



List - Prepend (L, x)

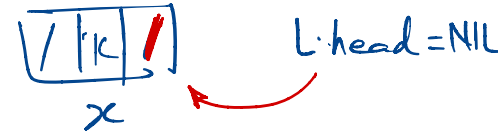
$x \cdot \text{next} = L \cdot \text{head}$

if $L \cdot \text{head} \neq \text{NIL}$

$L \cdot \text{head} \cdot \text{prev} = x$

$L \cdot \text{head} = x$

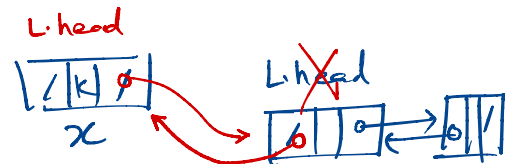
If list is empty:



$L \cdot \text{head} = \text{NIL}$

$L \cdot \text{head}$

if list is not empty:



List - insert (x, y)

: Insert node x after node y

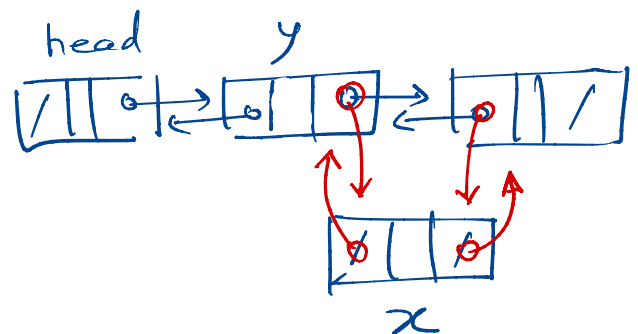
$x \cdot \text{next} = y \cdot \text{next}$

$x \cdot \text{prev} = y$

if $y \cdot \text{next} \neq \text{NIL}$

$y \cdot \text{next} \cdot \text{prev} = x$

$y \cdot \text{next} = x$



- Deleting a node from a linked list L

→ removes an element x from L

→ Splices x out of the list by updating pointers

→ First call list-search to retrieve x for a given key

List-Delete (L, x)

if $x.\text{prev} \neq \text{NIL}$

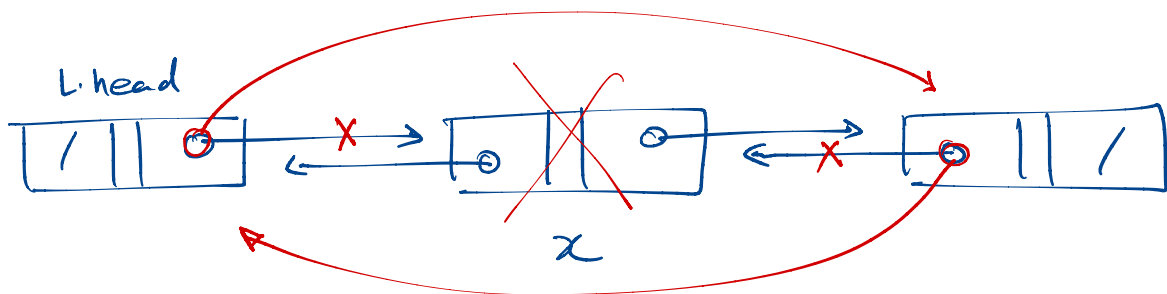
$x.\text{prev}.\text{next} = x.\text{next}$

else

$L.\text{head} = x.\text{next}$

if $x.\text{next} \neq \text{NIL}$

$x.\text{next}.\text{prev} = x.\text{prev}$



→ Sentinels

→ a dummy object that allows simplifying boundary conditions

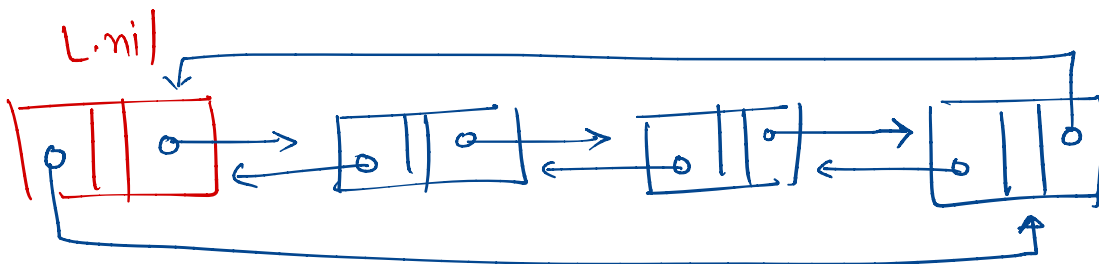
→ object $L.nil$ that represents NIL but also contains other attributes of an LL node.

→ Replace references to NIL by $L.nil$

→ Turn our doubly linked list into a Circular doubly linked list

→ Sentinel $L.nil$ lies between head & tail

⇒ $L.nil.next$: points to head of list
 $L.nil.prev$: points to tail of list



List-Delete' (L, x)

$x.prev.next = x.next$

$x.next.prev = x.prev$

List-Delete (L, x)

if $x.prev \neq NIL$
 $x.prev.next = x.next$

else $L.head = x.next$

if $x.next \neq NIL$
 $x.next.prev = x.prev$

Old list-Delete without a sentinel

List-Search' (L, k)

$L.nil.key = k$

$x = L.nil.next$

While ($x.key \neq k$)

$x = x.next$

if $x = L.nil$

return NIL

else return x

List-Insert' (x, y)

$x.next = y.next$

$x.prev = y$

$y.next.prev = x$

$y.next = x$

List-insert (x, y)

$x.next = y.next$

$x.prev = y$

if $y.next \neq NIL$

$y.next.prev = x$

$y.next = x$

Previous List-Insert
without Sentinel

— No prepend function needed. In order to prepend at head, use $y = L.nil$

Todo:
Verify the claim!

→ Reversing a doubly linked list

↳ Swap prev & next pointers

↳ Assign last node to head

List - Reverse (L)

$x = L.head$

While ($x \neq \text{NIL}$)

$tmp = x \rightarrow prev$

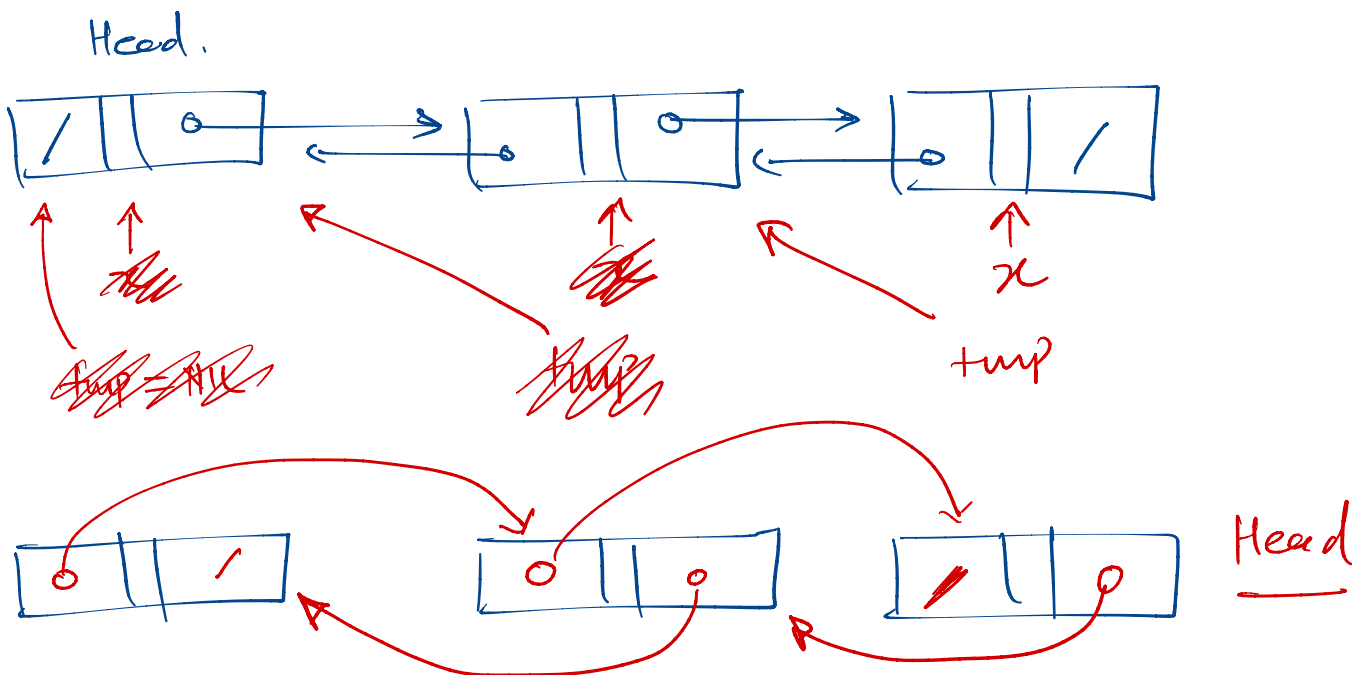
$x.prev = x.next$

$x.next = tmp$

$x = x.prev$

End While

$L.head = tmp.prev$



→ Joining two lists

↳ Given lists L_1 & L_2 , return a concatenation of L_1 & L_2 .

List-Join (L_1, L_2)

$L_1.\text{tail}.\text{next} = L_2.\text{head}$

$L_2.\text{head}.\text{prev} = L_1.\text{tail}$

return $L_1.\text{head}$
